

PyQCU 全库代码分析

Lattice QCD 的 Python/Cython 库：CUDA 加速 Dirac 算子与多重网格求解器

opencode analy

2026-08-14

摘要

本报告对 PyQCU 仓库全部代码进行证据驱动分析。PyQCU 是 Lattice QCD 的 Python/Cython 库：CUDA 加速的 Wilson/Clover Dirac 算子、BiStabCG 与多重网格 (MultiGrid) 求解器、stout smearing 与规范场生成，经 MPI 分布于 4D 进程网格。分析覆盖两层执行架构（纯 Python / C++ CUDA 后端 + Cython 桥）、三张扁平张量桥接协议 (`params/argv/set_ptrs`)、张量布局约定、新增的多线程多卡（一线程一卡）MultiGrid 路径、h5py 持久化体系与测试体系。核心结论：多线程多卡 MG 驱动已实现并验证（单卡 2/4 线程一致性 PASS），Cython 桥经 `nogil` 重构支持真并行；所有持久化统一 h5py 且多线程安全；散落的 Schur 算子与 33-tensor stencil 构建已合并进 `pyqcu/` 库代码。

目录

1	仓库概览	1
2	主题分析	2
2.1	两层执行层级架构	2
2.2	参数协议与调用生命周期	2
2.3	张量布局约定	2
2.4	多线程多卡（一线程一卡）MultiGrid 路径	2
2.5	HDF5 持久化 (h5py)	3
2.6	对象映射（代码符号 物理对象）	4
3	关键代码片段	4
3.1	多线程多卡驱动 (solve/worker 结构)	4
3.2	Cython 桥 nogil 模式	4
3.3	CudaSchurOp 多线程安全封装	4
3.4	h5py 多线程安全保存	5
3.5	多线程 33-tensor stencil 构建	5
4	参考源清单	6
5	结论与局限	7

1 仓库概览

类别	文件数	说明
Python (.py)	122	纯 Python 实现 + 测试 + Cython 源
C++ 头 (.h)	24	模板化 CUDA 内核 (cpp/cuda/qcu/include/)
CUDA 内核 (.cu)	30	内核实例化与启动封装 (cpp/cuda/qcu/src/)
文档 (.md)	134	各目录 AGENTS.md + docs/ + 开发报告
git 提交	1077	主分支 main (截至 2026-08-14)

git 元信息: HEAD = 148da33, 标签体系 stab<N>/dev<N>/bug<N> 独立编号。工作目录结构 (关键层级):

- 入口层: pyqcu/ (库)、examples/ (测试入口)、docs/ (文档)、logs/ (产物归档)
- 核心层: pyqcu/dslash/、pyqcu/solver/、pyqcu/smea/、pyqcu/tools/
- 桥接层: pyqcu/cuda/ (Cython 桥 qcu.pyx + 参数常量 define.py)
- 后端层: cpp/cuda/qcu/ (26 个模板头 + 30 个.cu + C API pyqcu.h)
- 兼容层: pyqcu/cann/ (昇腾 NPU)、cpp/{cann, dtk, maca}/ (占位)

2 主题分析

2.1 两层执行层级架构

PyQCU 的核心设计是**两层执行层级** (AGENTS.md:14-17):

- **纯 Python 层**: pyqcu/dslash/、pyqcu/solver/、pyqcu/smea/ 为 PyTorch 实现, 可跑 CPU/CUDA/昇腾 NPU (经 pyqcu.cann 兼容层)。约定所有 Python 代码 `import pyqcu.cann as _torch`, 不得直接 `import torch` —— NPU 不支持复数张量, cann 层自动分解实虚部。
- **C++ CUDA 后端**: cpp/cuda/qcu/ 为手写 CUDA 内核 + MPI halo 交换, 经 Cython 桥 pyqcu/cuda/qcu/qcu.pyx 暴露, 是生产路径。

联系与层次: Python 层 (算法参考/跨平台) 与 C++ 层 (生产性能) 通过 Cython 桥构成“参考实现 → 桥接 → 生产后端”的依赖链。多重网格求解器可**混合**两层: 最细层平滑用 C++ 后端 (`with_cuda_qcu=True`), 更粗层用纯 Python (pyqcu/solver/AGENTS.md)。

2.2 参数协议与调用生命周期

三个扁平张量桥接 Python C++ (pyqcu/cuda/define.py:11-90, 与 cpp/cuda/qcu/include/define.h 必须同步):

- `params` (`int32[54]`): 格点维度 `_LAT_X_=0`、网格大小、数据类型、迭代次数、计划选择、MG 层级配置
- `argv` (`float[7]`): `mass`、`atol`、`sigma`、MG 各层容差
- `set_ptrs` (`int64[100]`): C++ 运行时管理的 scratch 指针 (每个 `_SET_INDEX_` 槽位一个 `LatticeSet*`)

关键不变量: 调用生命周期 `applyInitQcu` → 操作 → `params[_SET_INDEX_] += 1` (每次调用间必须递增!) → `applyEndQcu` (`AGENTS.md:23`)。不递增导致 scratch 缓冲复用冲突、结果错误。`_SET_PLAN_` (`define.py:38`) 选择算子类型: -2 Laplacian、-1 Gauss gauge、0 Wilson dslash、1 BiStabCG/CG、2 Clover dslash。

2.3 张量布局约定

时空维永远是最后 4 轴 (...xyz), ward 索引负整数 (`wards['x']=-4`): 规范场 `[3,3,4,Lx,Ly,Lz,Lt]` (`color×color×dir×xyz`)、费米子场 `[4,3,Lx,Ly,Lz,Lt]` (`spin×color×xyz`)、Clover 项 `[4,3,4,3,Lx,Ly,Lz,Lt]`。HDF5 内部用 `zyxt` 序, 经 `ccdxyz2ccdptzyx/scxyz2psctzyx` 转换 (`pyqcu/tools/_define.py`)。奇偶预处理 (`ooxyz2pooxyz`) 沿最快变化维拆分, `p=0` 偶格点、`p=1` 奇格点。

2.4 多线程多卡 (一线程一卡) MultiGrid 路径

本次分析的重点。构成三件套:

(a) **Cython 桥真并行重构** (`pyqcu/cuda/qcu/qcu.pyx:1-16`): 全部桥函数改为 "GIL 段取指针 → with nogil 调 C++" 模式; 指针使用函数内局部 `cdef` 变量 (线程安全), 不再用模块级共享 `cdef`。pxd 声明 (`qcu.pxd`) 必须带 `nogil` 关键字, 且声明名不得与 `pyx` 内 `def` 同名 (否则被覆盖为 Python 函数导致 `nogil` 调用失败), 故新增 `qcu_api.pxd` 别名 `cimport X as _c_X`。

(b) **多线程安全 Schur 算子** (`pyqcu/cuda/_schur_op.py:47-94`): `CudaSchurOp` 封装 `applyCloverBistabCgDslas` (Schur 奇偶算子 $S = A_{oo} - k^2 D_{oe} A_{ee}^{-1} D_{eo}$)。每实例持 **独立** `params` 副本与 **独立** `set_ptrs` 副本, `_SET_INDEX_` 由线程安全槽位分配器 (`_SetIndexAllocator`, `_schur_op.py:27-45`) 独占分配。

(c) **多线程多卡驱动** (`pyqcu/cuda/_multi_gpu.py:133`): `MultiGpuMultigrid` 单进程内 `N` 线程 × 卡绑定并行。每线程 `torch.cuda.set_device` (CUDA current device 线程局部) + 独立 `params/argv/set_ptrs` 副本; 粗网格 Schur 算子 (33-tensor) 在主线程构建一次 (`build_schur_levels`, `_multi_gpu.py:45`), 各线程拷贝到本卡后填入自己的 `set_ptrs` 槽位。`verify_consistency` (`_multi_gpu.py:329`) 校验各线程解与参考解一致性。

层次归属: 驱动层 (`pyqcu/cuda/_multi_gpu.py`) → 算子层 (`_schur_op.py`) → 桥接层 (`qcu.pyx`) → 后端层 (C++ `LatticeSet`)。实测 (RTX 4060 单卡): 2 线程与 4 线程求解, 线程间最大相对差 $< 10^{-5}$ (PASS), 各线程 MG 最终残差 $\sim 9 \times 10^{-7}$ (`atol=1e-6`)。约束: 该驱动要求单 MPI rank (`mpirun -np 1`)——C++ `LatticeSet::init` 用 `COMM_WORLD` 真实 rank 覆盖 `_NODE_RANK_` (`lattice_set.h:102-105`); 多进程分布走每进程单线程实例路径。

2.5 HDF5 持久化 (h5py)

所有保存/读取统一 `h5py` (`pyqcu/tools/_io.py`):

- MPI 分布式路径: `grid000xyzt2hdf5000xyzt` (`_io.py:8`) 用 `driver='mpio'` 并行 I/O; 串行回退用 `gather/scatter`。
- 本地单张量路径: `save_tensor_h5/load_tensor_h5` (`_io.py:165/183`) ——每调用独立 File 句柄 (`with` 语句), **多线程安全**。
- null-vector/粗网格算子缓存: `build_schur_levels` 以 `.h5` 缓存 (`_multi_gpu.py:63-75`), 单句柄一次写入全部 dataset (避免逐 dataset 'w' 覆盖重建导致前序 dataset 丢失的反模式)。

多线程并发读写验证: 4 线程独立写/读往返逐元素相等、并发读同一文件一致(`pyqcu/testing/__init__.py:535`, 实测 PASS, `max_err=0`)。

2.6 对象映射 (代码符号 物理对象)

代码符号	对象概念	物理含义
hopping 项	Wilson 跃迁项	$\bar{\psi}(x)\gamma_{\mu}U_{\mu}(x)\psi(x+\hat{\mu})$, 夸克相邻格点跃迁
sitting 项	质量项	κ 相关对角项, Wilson 项 $(1 - \kappa \sum \dots)$
clover_term	Clover 项	$c_{sw}\sigma_{\mu\nu}F_{\mu\nu}$, $O(a)$ 改进
matvec_parity	Schur 奇偶算子	$S = A_{oo} - k^2 D_{oe} A_{ee}^{-1} D_{eo}$
null_vecs	近零空间向量	逆迭代 $v_i = v_i - A^{-1}Av_i$ 捕获低模
33-tensor stencil	粗网格算子	Galerkin $A_c = P^T S P$ (on-site+ 近邻 + 对角耦合)
restrict/prolong	层间转移	P/P^T 用局部正交化 null 向量
MultiGpuMultigrid	多卡 MG 驱动	一线程一卡并行求解

3 关键代码片段

3.1 多线程多卡驱动 (solve/worker 结构)

```
def solve(self):
    """并行求解: 每线程一个完整 C++ MG 流程。返回 {'threads': [...], 'results': ...}。"""
    from concurrent.futures import ThreadPoolExecutor
    main_dev = torch.device(f'cuda:{self.device_ids[0]}')
    # 主线程生成共享输入 (规范场/Clover/源) 与粗网格算子
    params_t, argv_t, set_ptrs_t = _clone_state()
    self._config_params(params_t, argv_t, set_ptrs_t, argv_t)
    g, fi, ce, cei, coo, coi, U_full, b_full, clover_full, kappa, av = \
        _setup_gpu_tensors(params_t, argv_t, set_ptrs_t, main_dev, self.mass,
```

3.2 Cython 桥 nogil 模式

```
def applyInitQcu(_set_ptrs, _params, _argv):
    cdef long long set_ptrs, params, argv
    set_ptrs = _set_ptrs.contiguous().data_ptr()
    params = _params.contiguous().data_ptr()
    argv = _argv.contiguous().data_ptr()
```

```
with nogil:
    _c_applyInitQcu(set_ptrs, params, argv)
```

3.3 CudaSchurOp 多线程安全封装

```
class CudaSchurOp(object):
    """C++ CUDA 实现的 Schur 奇偶算子 (matvec_parity 等价物) 。

    matvec(x_o) -> y_o: x_o/y_o 为 [12,X,Y,Z,T/2] 奇子格场。
    构造即分配独立 LatticeSet (scratch 缓冲) ; release() 释放。

    参数:
        av: argv 张量 (float, real dtype, 长度 7)
        g: gauge 场 [2,3,3,4,X,Y,Z,T/2] (奇偶拆分)
        ce/coo: Clover 偶偶/奇奇项
        cei/coi: Clover 偶偶/奇奇逆
        device: 绑定的 torch 设备 (多线程多卡模式下每线程一个 device)
    """

    def __init__(self, av, g, ce, coo, cei, coi, device=None):
        self.device = device if device is not None else torch.device('cuda')
        self.params = _module_params.clone()
        self.params[define._SET_INDEX_] = _GLOBAL_SET_ALLOC.next()
        self.set_index = int(self.params[define._SET_INDEX_])
        self.params[define._SET_PLAN_] = 1
```

3.4 h5py 多线程安全保存

```
def save_tensor_h5(tensor: torch.Tensor, file_name: str, dataset: str = 'data',
                   verbose: bool = False):
    """单张量 HDF5 保存 (h5py, 多线程安全) 。

    每次调用新建独立 File 句柄 (with 语句) , 不持有全局句柄;
    多线程各线程独立调用即可安全并发 (HDF5 线程安全模式) ,
    适用于 null-vector / 粗网格算子缓存等本地持久化。
    张量内部布局 xyzt 原样保存; 用户自行保证与加载端布局约定一致。
    """

    import numpy as np
    arr = tensor.detach().cpu().contiguous().numpy()
    with h5py.File(file_name, 'w') as f:
        f.create_dataset(dataset, data=arr)
    if verbose:
        print(f"PYQCU::TOOLS::IO:\n Tensor {tuple(tensor.shape)} saved to "
              f"{file_name} (dataset='{dataset}')")
```

3.5 多线程 33-tensor stencil 构建

```
def build_stencil_mt(matvec_ops, lonv, E, e, lat_fine_odd, lat_coarse_odd,
                    dt, device, nthreads=4, verbose=True):
    """多线程 33-tensor stencil build. matvec_ops: 每线程一个 matvec 算子 (如 CudaSchurOp 列表) 。
```

各线程探测点写集不相交（probe_point 写 sit/hop_nn/hop_diag 的不同坐标片），线程安全；适合多卡（一线程一卡）场景用每线程独立 GPU 算子并行构建。

```

"""
from concurrent.futures import ThreadPoolExecutor
import time
Xc, Yc, Zc, Tc = lat_coarse_odd
Nc = Xc * Yc * Zc * Tc
dims = [Xc, Yc, Zc, Tc]
sit = torch.zeros([E, E, Xc, Yc, Zc, Tc], dtype=dt, device=device)
hop_nn = torch.zeros([2, 4, E, E, Xc, Yc, Zc, Tc], dtype=dt, device=device)
hop_diag = torch.zeros([2, 2, 6, E, E, Xc, Yc, Zc, Tc], dtype=dt, device=device)
t0 = time.perf_counter()
chunk = (Nc + nthreads - 1) // nthreads

def worker(tid):
    op = matvec_ops[tid % len(matvec_ops)]
    c0 = tid * chunk
    c1 = min(Nc, c0 + chunk)
    for c_idx in range(c0, c1):
        for ee in range(E):
            _probe_point(op, lonv, E, ee, c_idx, sit, hop_nn, hop_diag, dims, Nc)

with ThreadPoolExecutor(max_workers=nthreads) as ex:
    list(ex.map(worker, range(nthreads)))

```

4 参考源清单

文件: 行号	类型	内容/作用
AGENTS.md:14-17	仓库	两层执行层级定义
pyqcu/cuda/define.py:11-90	仓库	参数索引常量（54 项）
pyqcu/cuda/qcu/qcu.pyx:1-16	仓库	Cython 桥 nogil 重构
pyqcu/cuda/qcu/qcu_api.pxd	仓库	pxd 别名声明（nogil 调用）
pyqcu/cuda/_schur_op.py:27-94	仓库	CudaSchurOp 多线程封装
pyqcu/cuda/_multi_gpu.py:45-331	仓库	多线程多卡 MG 驱动
pyqcu/tools/_multigrid.py:277-404	仓库	33-tensor stencil build（合并迁移）
pyqcu/tools/_io.py:165-200	仓库	h5py 多线程安全读写
pyqcu/testing/__init__.py:535-616	仓库	多线程验证测试
cpp/cuda/qcu/include/lattice_set.h:102-105	仓库	COMM_WORLD rank 覆盖
cpp/cuda/qcu/include/lattice_clover_multigrid.h	仓库	C++ Clover MG（5-stream）
pyqcu/cuda/_multi_gpu.py:63-75	仓库	h5py 粗算子缓存（单句柄）

5 结论与局限

结论

主要结论：

1. PyQCU 是”纯 Python 参考层 + C++ CUDA 生产层”的双层格点 QCD 库，参数协议与 `_SET_INDEX_` 递增不变量是全库正确性的基石。
2. 多线程多卡（一线程一卡）MultiGrid 路径已完备：Cython 桥 `nogil` 真并行、每线程独立状态副本、粗算子共享只读；单卡 2/4 线程一致性验证 PASS ($< 10^{-5}$)。
3. 持久化统一 `h5py`：MPI 并行 I/O + 多线程安全的本地 `save/load_tensor_h5`。
4. 必要 python 代码已合并进 `pyqcu/`：`CudaSchurOp`、33-tensor stencil build、多线程多卡 MG 驱动、`h5py` 缓存。

局限与边界：

- 本机仅 1 张 RTX 4060 (8GB)，”一线程一卡”验证为单卡多线程（线程隔离与结果一致性等价判据）；真多卡（多张物理卡）部署需在多 GPU 节点实测。
- `MultiGpuMultigrid` 要求单 MPI rank；MPI 多进程分布与多线程组合暂不支持（C++ `LatticeSet` 用 `COMM_WORLD` rank 覆盖）。
- 本报告为只读分析，行号基于 2026-08-14 的仓库状态；未对 `logs/` 归档产物与 `refer/` 历史报告做深度代码级分析。